Chapter Six

Client-Side Scripting (JavaScript)

Introduction to JavaScript

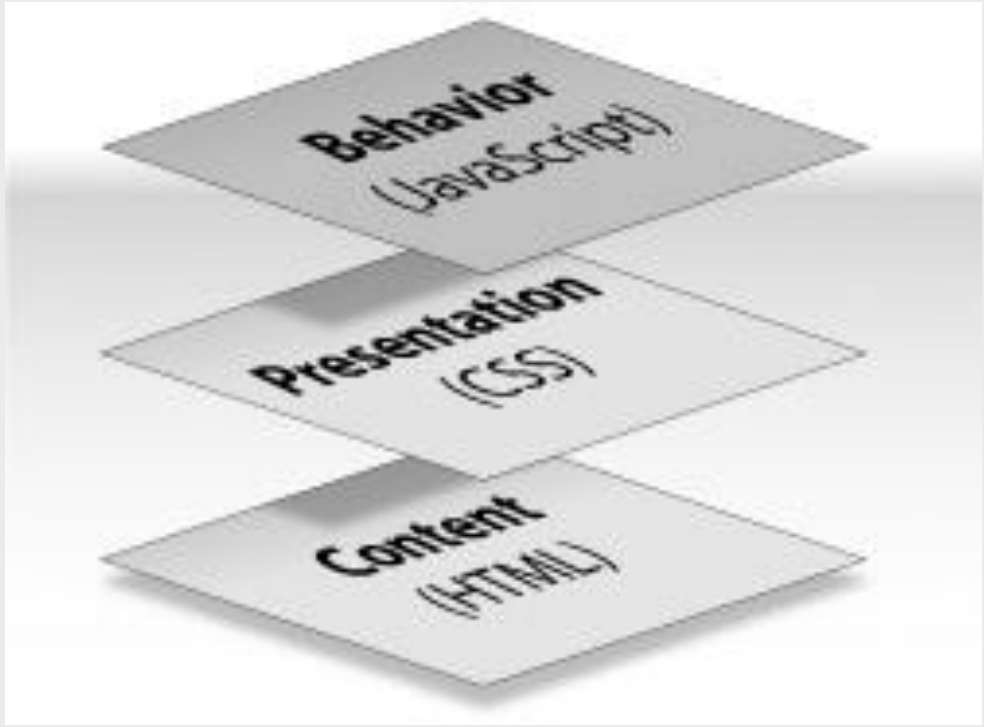
- ✓ It was created in 1995 by Brendan Eich while he was an engineer at Netscape.
- ✓ JavaScript is a scripting language that empowers web developers to create interactive websites.
  - ❑ Scripting language is a series of commands that are able to be executed without the need for compiling.
- ✓ Although it shares many of the features and structures of the full Java language, it was developed independently.
- ✓ Javascript can interact with HTML source code, enabling Web authors to spice up their sites with dynamic content.

...cont'd

- ✓ JavaScript is a lightweight, cross-platform, single-threaded, and interpreted compiled programming language
- ✓ JavaScript can be used for Client-side developments as well as Server-side developments.
- ✓ JavaScript is a versatile programming language that offers numerous career opportunities in the field of:
  - Web Development—React, Angular, and Vue.js
  - Mobile Applications—React Native and NativeScript
  - Game Development — PixiJS, Phaser, BabylonJS
  - IoT (Internet of Things)— Node.js

...cont'd

- ✓ Why use JavaScript?
  - ❑ To create dynamic and interactive Web pages
    - ➢ To reduce the burden of server.
      - ☛ JavaScript runs in the browser, not on the Web server.
  - ❑ Better performance
    - ➢ It validates the data that users enter into the form, before it is sent to your Web application.



- ✓ How Does It Work?
  - ❑ Embedded within HTML page
  - ❑ Executes on client machine—Fast, no connection needed once loaded
  - ❑ Interpreted (not compiled)
    - ➢ No special tools required

...cont'd

- JavaScript allows interactivity such as:
  - Implementing form validation
  - Enhancing user interactions
  - React to user actions, e.g. handle keys
  - Changing an image on moving mouse over it
  - Sections of a page appearing and disappearing
  - Content loading and changing dynamically
  - Performing complex calculations
  - Custom HTML controls, e.g. scrollable table
  - Implementing AJAX functionality

JavaScript Basic

- You should place all your JavaScript code within <script> tags

  <script>…</script>

- You can place the <script> tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the <head> tags.
- The <script> tag alerts the browser program to start interpreting all the text between these tags as a script.
- A simple syntax of your JavaScript will appear as follows.

```
<script ...>
   JavaScript code
</script>
```

...cont'd

- The script tag takes two important attributes –
  - **Language** – This attribute specifies what scripting language you are using.
    - Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
  - **Type** – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".
    - JavaScript has become default lannguage in HTML5, and modern browsers, so now adding type is not required.

```
<script language = "javascript" type = "text/javascript">
   JavaScript code
</script>
```

...cont'd

- ✓ Adding JavaScript to HTML Pages
  - **Embedding code**

```
<script type="text/javascript">
    // Your JavaScript code goes here
</script>
```

  - **Inline code**

```
<a href="#" onClick="alert('Welcome!');">Click Me</a>
```

  - **External file**

```
<script src="xyz.js"> </script>
```

...cont'd

```
<html>
<head>
   <title> Your first JavaScript program </title>
<head>
<body>
   <script language = "javascript" type = "text/javascript">
      document.write("Hello World!")
   </script>
</body>
</html>
```

This code produce the output on an HTML page:
Hello World!

Basic JavaScript Input Output

- In JavaScript, there are several ways to handle user input and output, allowing your programs to interact with users.

Input

- **Prompt:** The prompt() function displays a dialog box where the user can enter text. It returns the entered text as a string.

```
let name = prompt("What is your name?");
console.log("Hello, " + name + "!");
```

- **Confirm:** The confirm() function displays a dialog box with a message and two buttons (usually "OK" and "Cancel"). It returns true if the user clicks "OK" and false otherwise.

```
let confirmed = confirm("Are you sure you want to continue?");
if (confirmed) {
console.log("Continuing...");
} else {
 console.log("Action cancelled.");
}
```

...cont'd

Output

- **Console.log:** This is the most common way to print output to the browser's developer console for debugging or logging purposes.

```
console.log("This message will be displayed in the console.");
let age = 30;
console.log("Your age is:", age);
```

- **Alert:** The alert() function displays a modal dialog box with a message. The user has to click "OK" to dismiss it.

```
alert("This is an alert message!");
```

JavaScript Keywords

- ✓ JavaScript statements often start with a keyword to identify the JavaScript action to be performed.
- ✓ Here is a list of some of the keywords

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JavaScript Identifiers

- All JavaScript variables must be identified with unique names.
- These unique names are called identifiers.
- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).
- The general rules for constructing names for variables (unique identifiers) are:
  - Names can contain letters, digits, underscores, and dollar signs.
  - Names must begin with a letter.
  - Names can also begin with $ and _ (but we will not use it in this tutorial).
  - Names are case sensitive (y and Y are different variables).
  - Reserved words (like JavaScript keywords) cannot be used as names.
  - JavaScript identifiers are case-sensitive.

JavaScript Data Types

- ✓ JavaScript defines various data types like numbers, strings, booleans, and objects, each with specific ways to represent and manipulate data.
- ✓ JavaScript provides different datatypes to hold different values on variables.
- ✓ JavaScript is a dynamic programming language, which means do not need to specify the type of variable.
- ✓ There are two types of data types in JavaScript.
  - ❑ Primitive data type: They can hold a single simple value.
  - ❑ Non-primitive (reference) data type: They can hold multiple values.

...cont'd

- ✓ A variable data type is not declared explicitly but rather implicitly according to its initial value assignment.
- ✓ Also unique to JavaScript, there is no explicit distinction between integers and real-valued numbers.
- ✓ There are four specific data types in JavaScript: numbers, strings, Booleans, and null values.

Type	Description	Examples
number	Any number without quotes	`42` or `16.3` or `2e-16`
string	A series of characters enclosed in quote marks	`"Hello!"` or `"10"` or `''` or `""`
Boolean	A logical value	`true` or `false`
null	A keyword meaning no value	`null`

...cont'd

```
// It store string data type
let txt = “Hello World";
// It store integer data type
let a = 5;
// It store Boolean data type
(a == b )
// To check Strictly (i.e. Whether the datatypes
//of both variables are same)
(a === b)---> returns true to the console
// It store array data type
let places= ["GFG", "Computer", "Hello"];
```

```
// It store object data (objects are  represented in the below way mainly)
let Student = {
    firstName: "Johnny",
    lastName: "Diaz",
    age: 35,
    mark: "blueEYE"
}
```

JavaScript Variables

- ✓ Variables are Containers for Storing Data
- ✓ Each variable is identified by a variable name, also known as an identifier.
- ✓ Each variable name is associated with a specific memory location, and the interpreter uses it to determine its location.
- ✓ JavaScript Variables can be declared in 4 ways:
  - Automatically —variableName = anyValue
  - Using **var** —var variableName1 = initialValue1 , variableName2 = initialValue2, ...
  - Using **let** —let variableName = initialValue;
  - Using **const** —variableName = anyValue

...cont'd

- Type Conversion
  - var myVar = 12
  - myVar = "university"
- Mixing Strings and Numbers
  - 8 + 8 ☞ 16
  - "8" + 8 ☞ "88"
  - 8 + "8" ☞ "88"
  - "8" + "8" ☞ "88"
  - 8 + 8 + "8" ☞ "168"
  - 8 + "8" + 8 ☞ "888“
  - 8 + (8 + "8") ☞ "888"

var	let	const
Global scoped or function scoped	block scope	the same as the let keyword, except the user cannot update it
It can be updated and re-declared in the same scope.	It can be updated but cannot be re-declared in the same scope.	It can neither be updated or re-declared in any scope.

Operators

- An operator is simply a symbol that tells the compiler (or interpreter) to perform a certain action.
  - Arithmetic operators
  - Logical operators
  - Comparison operators
  - String operators
  - Bit-wise operators
  - Assignment operators
  - Conditional operators

Precedence	Operator
1	Parentheses, function calls
2	, ~, -, ++, --, new, void, delete
3	*, /, %
4	+, -
5	<<, >>, >>>
6	<, <=, >, >=
7	==, !=, ===, !==
8	&
9	^
10	|
11	&&
12	||
13	?:
14	=, +=, -=, *=, …
15	The comma (,) operator

...cont'd

- ✓ The === operator in JavaScript is the strict equality operator.
- ✓ It's used to compare two values and determine if they are exactly the same, considering both their data type and value.
  - ❑ Compares both data type and value
  - ❑ Returns true only for exact matches: If the data types and values are identical, === returns true. Otherwise, it returns false.

```
console.log(1 === 1);          // true (same data type and value)
console.log("1" === 1);        // false (different data types: string vs number)
console.log(true === "true");  // false (different data types: boolean vs string)
console.log(null === undefined); // false (different data types)
console.log(0 === -0);         // true (same data type and value, even for -0)
```

Scope of Variable

- When you use a variable in a JavaScript program that uses functions.
  - **global scope** variable is one that is declared outside a function and is accessible in any part of your program
    - it's written outside of a function.
  - **local variable** is declared inside a function and stops existing when the function ends
  - **Block scope**
    - Blocks are used to group a single statement or a set of statements together.
    - You can use the const or let keywords to declare a block-scope local variable.
    - Note that you can't use the var keyword to declare variables with block scope.

JavaScript Statements

- ✓ JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments.
- ✓ Most JavaScript programs contain many JavaScript statements.
- ✓ The statements are executed, one by one, in the same order as they are written.
- ✓ Semicolons separate JavaScript statements.
- ✓ When separated by semicolons, multiple statements on one line are allowed
- ✓ JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.
- ✓ JavaScript statements can be grouped together in code blocks

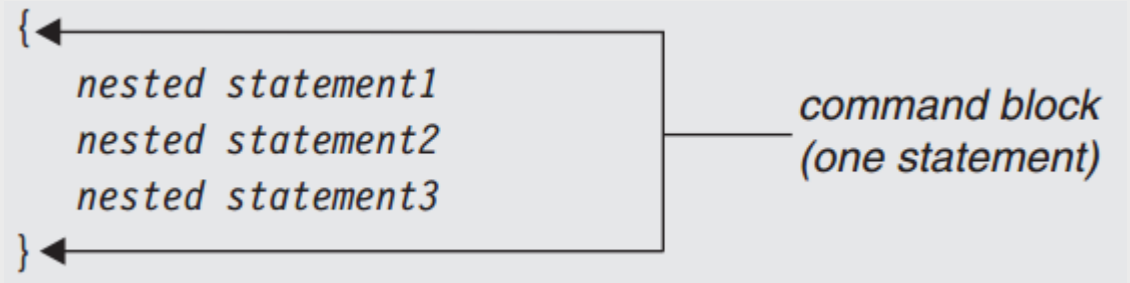
...cont'd

- Multiple statements
  - The JavaScript interpreter accepts multiple statements on the same line.
    - statement1 ; statement2; statement3; ...

```
statement1
statement2
statement3
```

```
statement1 ;
statement2;
statement3;
```

- Nested Statemens
  - A command block is a unit of statements enclosed by curly braces.



Control Structures

- **Conditional statements** are key to all programming languages.
  - They allow your program (or script in this case) to execute different code segments based on varying conditions.
  - if, if---else, nested if--else
- **Loop Statements** are control structures that perform a set of actions more than once.
  - for loop, while loop and do---while loop

Array in JavaScript

- ✓ The Array object is used to store multiple values in a single variable.
- ✓ Arrays in JavaScript are represented by the Array Object, we need to "new Array()" to construct this object
- ✓ The first element of the array is "Array[0]" until the last one Array[i-1].
  - ❑ E.g. myArray = new Array(5)
  - ❑ We have myArray[0,1,2,3,4].

```
<script language="JavaScript">
        student = new Array(3);
         student [0] = "Bekele";
         student[1] = "Lemma";
         student[2] = "Helen";
        document.write(student[0] + "<br>");
        document.write(student[1] + "<br>");
        document.write(student[2] + "<br>");
</script>
```

JavaScript Array Methods

Name	Description
new Array()	Creates a new Array
at()	Returns an indexed element of an array
concat()	Joins arrays and returns an array with the joined arrays
constructor	Returns the function that created the Array prototype
copyWithin()	Copies array elements within the array, to and from specified positions
entries()	Returns a key/value pair Array Iteration Object
every()	Checks if every element in an array pass a test
fill()	Fill the elements in an array with a static value
filter()	Creates a new array with every element in an array that pass a test
find()	Returns the value of the first element in an array that pass a test
findIndex()	Returns the index of the first element in an array that pass a test
findLast()	Returns the value of the last element in an array that pass a test
findLastIndex()	Returns the index of the last element in an array that pass a test
flat()	Concatenates sub-array elements

- new Array
- flatMap()
- forEach()
- from()
- includes()
- indexOf()
- isArray()
- join()
- keys()
- lastIndexOf()
- length
- map()
- of()
- pop()
- prototype
- push()

- reduce()
- reduceRight()
- reverse()
- shift()
- slice()
- some()
- sort()
- splice()
- toReversed()
- toSorted()
- toSpliced()
- toString()
- unshift()
- valueOf()
- with()

Examples

```
// Create an Array
const cars = new Array(["Saab", "Volvo", "BMW"]);
```

```
// Add Values to the Set
cars.push("Saab");
cars.push("Volvo");
cars.push("BMW");
```

Create an array without the new Array() method:

```
// Create an Array
const cars = ["Saab", "Volvo", "BMW"];
```

Get the third element of fruits:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
let fruit = fruits.at(2);
//OR
let fruit = fruits[2];
```

```
//Join three arrays:

const arr1 = ["Cecilie", "Lone"];
const arr2 = ["Emil", "Tobias", "Linus"];
const arr3 = ["Robin"];
const children = arr1.concat(arr2, arr3);
```

```
//Calls a function for each element in fruits:

const fruits = ["apple", "orange", "cherry"];
fruits.forEach(myFunction);
```

JavaScript Function

- ✓ Function is a block of organized reusable code (a set of statements) for performing a single or related action
- ✓ contains some code that will be executed only by an event or by a call to that function
  - ❑ To keep the browser from executing a script as soon as the page is loaded, you can write your script as a function

1. **Built-in Functions** provided by the language and you cannot change them to suit your needs.
2. **A user-defined** function name that should be unique,
   - ➢ use the keyword function, separated by the name of the function and a list of parameters enclosed within parentheses and separated by commas,
   - ➢ A list of statements composing the body of the function enclosed within curly braces {}.

...cont'd

- There are three ways of writing a function in JavaScript:
  - Function Declaration: declares a function with a function keyword.

```
function getSalary(paramA, paramB) {
    // Set of statements
}
```

  - Function Expression: It is similar to a function declaration without the function name.

```
let total= function(paramA, paramB){
    // Set of statements
}
```

  - Functions as Variable Values: Functions can be used the same way as you use variables.

```
// Function to convert Fahrenheit to
Celsius
function toCelsius(fahrenheit) {
    return (fahrenheit - 32) * 5/9;
  }
```

...cont'd

- ✓ Example : A basic javascript function, here we create a function that divides the 1st element by the second element.
  - use the keyword function, separated by the name of the function and parameters within parenthesis.
  - The part of the function inside the curly braces {} is the body of the function.

```
function myFunction(g1, g2) {
    return g1 / g2;
}
const value = myFunction(8, 2); // Calling the function
console.log(value);
```

...cont'd

✓ Arrow Function:

- It is one of the most used and efficient methods to create a function in JavaScript because of its comparatively easy implementation.
- It is a simplified as well as a more compact version of a regular or normal function expression or syntax.

```
let function_name = (argument1, argument2 ,..) => expression
```

```
//This example describes the usage of the Arrow function.
const a = ["Hydrogen", "Helium", "Lithium", "Beryllium"];
const a2 = a.map(function (s) {
    return s.length;
});
console.log("Normal way ", a2); // [8, 6, 7, 9]
const a3 = a.map((s) => s.length);
console.log("Using Arrow Function ", a3); // [8, 6, 7, 9]
```

JavaScript Events

- are actions that occur as a result of browser activities or user interactions with the web pages, such as
  - user performs an action (mouse click or enters data)
  - validate the data entered by a user in a web form
  - Communicate with Java applets and browser plug-ins
- Event categories
  - Keyboard and mouse events
  - Load events
  - Form-related events
    - onFocus() refers to placing the cursor into the text input in the form.
  - Others
    - Errors, window resizing.

Events defined by JavaScript

HTML elements	HTML tags	JavaScript defined events	Description
Link	<a>	click dblClick mouseDown mouseUp mouseOver	Mouse is clicked on a link Mouse is double-clicked on a link Mouse button is pressed Mouse button is released Mouse is moved over a link
Image	<img>	load abort error	Image is loaded into a browser Image loading is abandoned An error occurs during the image loading
Area	<area>	mouseOver mouseOut dblClick	mouse is moved over an image map area mouse is moved from image map to outside mouse is double-clicked on an image map
Form	<form>	submit Reset	user submits a form user refreshes a form

Event Handlers

- ✓ When an event occurs, a code segment is executed in response to a specific event is called “event handler”
- ✓ Event handler names are quite similar to the name of events they handle
- ✓ E.g event handler for the “click” event is “onClick”.
- ✓ <HTMLtag eventhandler=“JavaScript Code”>

```
<!DOCTYPE html>
<html>
<body>
<h2>The onclick Attribute</h2>
<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>
<p id="demo"></p>
</body>
</html>
```

...cont'd

Event Handlers	Triggered when
onChange	The value of the text field, textarea, or a drop down list is modified
onClick	link, an image or a form element is clicked once
onDblClick	The element is double-clicked
onMouseDown	The user presses the mouse button
onLoad	A document or an image is loaded
onSubmit	A user submits a form
onReset	The form is reset
onUnLoad	The user closes a document or a frame
onResize	A form is resized by the user

...cont'd

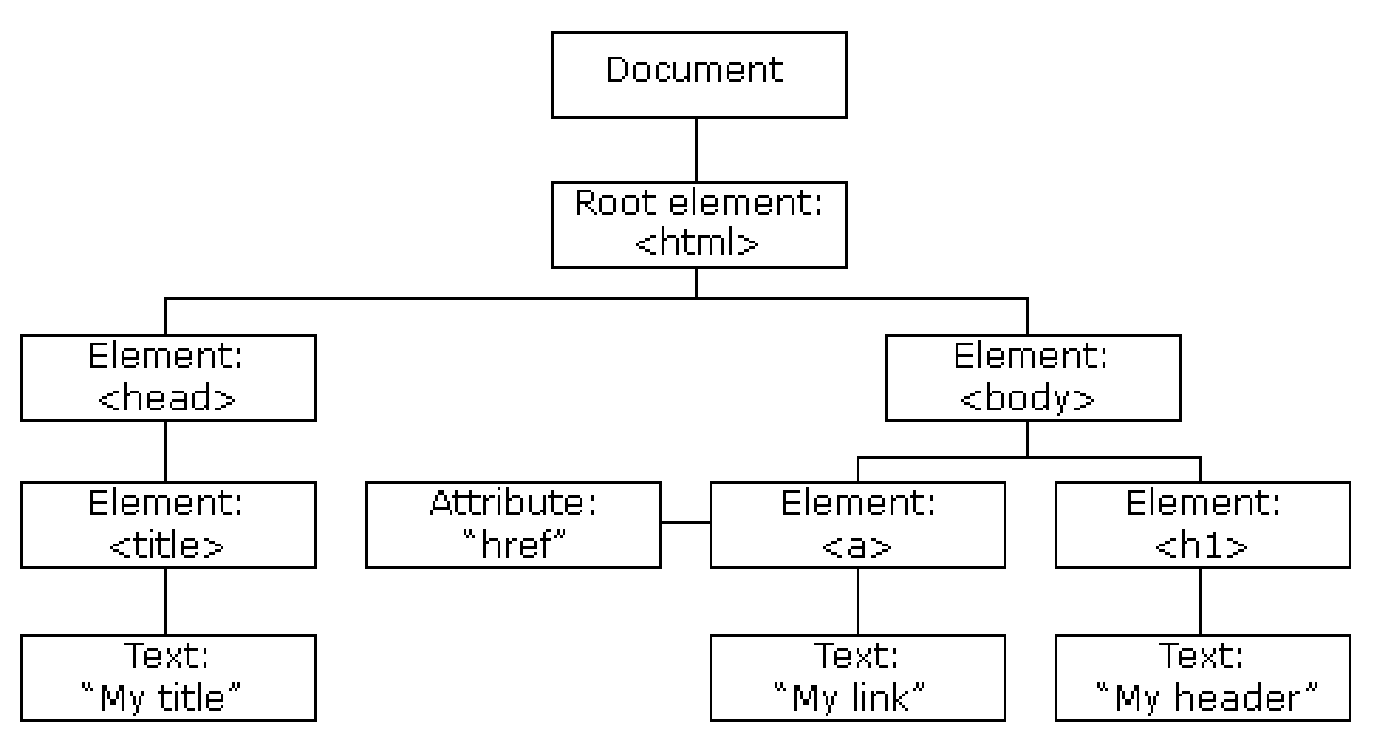
```
<html>
<head><title>onClick Event Handler Example</title>
<script language="JavaScript">
function warnUser(){
    return confirm("CSIT students?");
}
</script>
</head>
<body>
<a  href="index.html",  onClick="return  warnUser();">CSIT  Students  access only</a>
</body>
</html>
```

JavaScript DOM (Document Object Model)

- ✓ DOM is a programming interface for HTML and XML documents.
- ✓ It defines the logical structure of documents and the way a document is accessed and manipulated.
- ✓ The DOM is a W3C (World Wide Web Consortium) standard and it defines a standard for accessing documents.
- ✓ The W3C DOM standard is divided into three different parts:
  - Core DOM – standard model for all document types
  - XML DOM – standard model for XML documents
  - HTML DOM – standard model for HTML documents

...cont'd

- ✓ When a web page is loaded, the browser creates a Document Object Model of the page.
- ✓ The HTML DOM model is constructed as a tree of Objects:



...cont'd

- With HTML DOM, we can easily access and manipulate tags, IDs, classes, attributes, or elements of HTML using commands or methods provided by the document object.
- Using DOM JavaScript we get access to HTML as well as CSS of the web page and can also modify the behavior of the HTML elements.
- DOM is basically the representation of the same HTML document but in a tree-like structure composed of objects.
- JavaScript can not understand the tags(<h1>H</h1>) in HTML document but can understand object h1 in DOM.
- **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

Methods of Document Object

- ✓ DOM provides various methods that allows users to interact with and manipulate the document.
- ✓ HTML DOM properties are values (of HTML Elements) that you can set or change.
- ✓ The programming interface is the properties and methods of each object.
  - ❑ A property is a value that you can get or set (like changing the content of an HTML element).
  - ❑ A method is an action you can do (like add or deleting an HTML element).

```
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>
```

DOM Method
- to access an HTML element

DOM property
- to get the content of an element

...cont'd

- Some commonly used DOM methods are:
  - **write("string"):** Writes the given string on the document.
  - **getElementById()**: returns the element having the given id value.
  - **getElementsByName()**: returns all the elements having the given name value.
  - **getElementsByTagName()**: returns all the elements having the given tag name.
  - **getElementsByClassName()**: returns all the elements having the given class name.

```
//This example returns a list of all elements with class="intro".
const x = document.getElementsByClassName("intro");
```

...cont'd

- ✓ Changing HTML Elements

Property	Description
element.innerHTML = new html content	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.style.property = new style	Change the style of an HTML element
element.setAttribute(attribute, value)	Change the attribute value of an HTML element

- ✓ Adding and Deleting Elements

Method	Description
document.createElement(element)	Create an HTML element
document.removeChild(element)	Remove an HTML element
document.appendChild(element)	Add an HTML element
document.replaceChild(new, old)	Replace an HTML element
document.write(text)	Write into the HTML output stream

...cont'd

✓ Adding Events Handlers

Method	Description
document.getElementById(id).onclick = function(){code}	Adding event handler code to an onclick event

✓ Finding HTML Objects

Property	Description
document.anchors	Returns all <a> elements that have a name attribute
document.body	Returns the <body> element
document.forms	Returns all <form> elements
document.head	Returns the <head> element
document.images	Returns all <img> elements
document.links	Returns all <area> and <a> elements that have a href attribute

...cont'd

- ✓ JavaScript HTML DOM Elements
  - ❑ Ways of finding HTML elements in order to manipulate HTML elements
    - ➢ Finding HTML elements by id ☞ document.getElementById("intro");
    - ➢ Finding HTML elements by tag name ☞ document.getElementsByTagName("p");
    - ➢ Finding HTML elements by class name ☞ document.getElementsByClassName("intro");
    - ➢ Finding HTML elements by CSS selectors ☞ document.querySelectorAll("p.intro");
    - ➢ Finding HTML elements by HTML object collections

```
const x = document.forms["frm1"];
let text = "";
for (let i = 0; i < x.length; i++) {
  text += x.elements[i].value + "<br>";
}
document.getElementById("demo").innerHTML = text;
```

Changing HTML Content

- ✓ The easiest way to modify the content of an HTML element is by using the innerHTML property.
- ✓ To change the content of an HTML element, use this syntax:
  - document.getElementById(id).innerHTML = new HTML

```
<html>
<body>
<p id="p1">Hello World!</p>
<script>
document.getElementById("p1").innerHTML = "New text!";
</script>
</body>
</html>
```

Changing the Value of an Attribute

- To change the value of an HTML attribute, use this syntax:
  - document.getElementById(id).attribute = new value

```
<!DOCTYPE html>
<html>
<body>
<img id="myImage" src="smiley.gif">
<script>
document.getElementById("myImage").src =
"landscape.jpg";
</script>
</body>
</html>
```

A JavaScript changes the src attribute of that element from "smiley.gif" to "landscape.jpg"

Dynamic HTML content

- JavaScript can create dynamic HTML content:
  - Date : Thu May 23 2024 10:07:15 GMT+0300 (East Africa Time)

```
<!DOCTYPE html>
<html>
<body>
<script>
document.getElementById("demo").innerHTML = "Date : " + Date(); </script>
</body>
</html>
```

...cont'd

- document.write()
  - In JavaScript, document.write() can be used to write directly to the HTML output stream:

```
<!DOCTYPE html>
<html>
<body>
<p>Hello World</p>
<script>
document.write(Date());
</script>
<p>Welcome</p>
</body>
</html>
```

Never use document.write() after the document is loaded. It will overwrite the document.

JavaScript Forms

- ✓ HTML form validation can be done by JavaScript.
- ✓ If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

```
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
```

The function can be called when the form is submitted:

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
Name: <input type="text" name="fname">
<input type="submit" value="Submit">
</form>
```

...cont'd

- Data Validation
  - Data validation is the process of ensuring that user input is clean, correct, and useful.
  - Validation can be defined by many different methods, and deployed in many different ways.
    - **Server side validation** is performed by a web server, after input has been sent to the server.
    - **Client side validation** is performed by a web browser, before input is sent to a web server.
  - **HTML Constraint Validation**
    - HTML5 introduced a new HTML validation concept called constraint validation.
    - HTML constraint validation is based on:
      - Constraint validation HTML Input Attributes
      - Constraint validation CSS Pseudo Selectors
      - Constraint validation DOM Properties and Methods

...cont'd

```
<html><head>
<title>Form Example</title>
<script LANGUAGE="JavaScript">
function validate() {
    if (document.form1.yourname.value.length < 1)
{
        alert("Please enter your full name.");
        return false;
    }
   if (document.form1.address.value.length < 3) {
       alert("Please enter your address.");
       return false;
   }
   if (document.form1.phone.value.length < 3) {
       alert("Please enter your phone number.");
       return false;
   }
   return true;
}
</script>
</head>
<body>
<h1>Form Example</h1>
<p>Enter the following information. When you press the Display button, the data
you entered will be validated, then sent by email.</p>
<form name="form1" action="mailto:user@host.com" enctype="text/plain“
onSubmit="validate();">
<p><b>Name:</b> <input type=“text" length="20" name="yourname"></p>
<p><b>Address:</b> <input type=“text" length="30" name="address"></p>
<p><b>Phone: </b> <input type=“text" length="15" name="phone"></p>
<p><input type=“submit" value="Submit"></p>
</form></body></html>
```

...cont'd

- ✓ Constraint Validation HTML Input Attributes

Attribute	Description
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

...cont'd

- ✓ Constraint Validation CSS Pseudo Selectors

Selector	Description
:disabled	Selects input elements with the "disabled" attribute specified
:invalid	Selects input elements with invalid values
:optional	Selects input elements with no "required" attribute specified
:required	Selects input elements with the "required" attribute specified
:valid	Selects input elements with valid values

Changing CSS

- The HTML DOM allows JavaScript to change the style of HTML elements.
- To change the style of an HTML element, use this syntax:
  - document.getElementById(id).style.property = new style
  - The following example changes the style of a <p> element:

```
<p id="p2">Hello World!</p>
<script>
document.getElementById("p2").style.color = "blue";
</script>
```

JavaScript HTML DOM Events

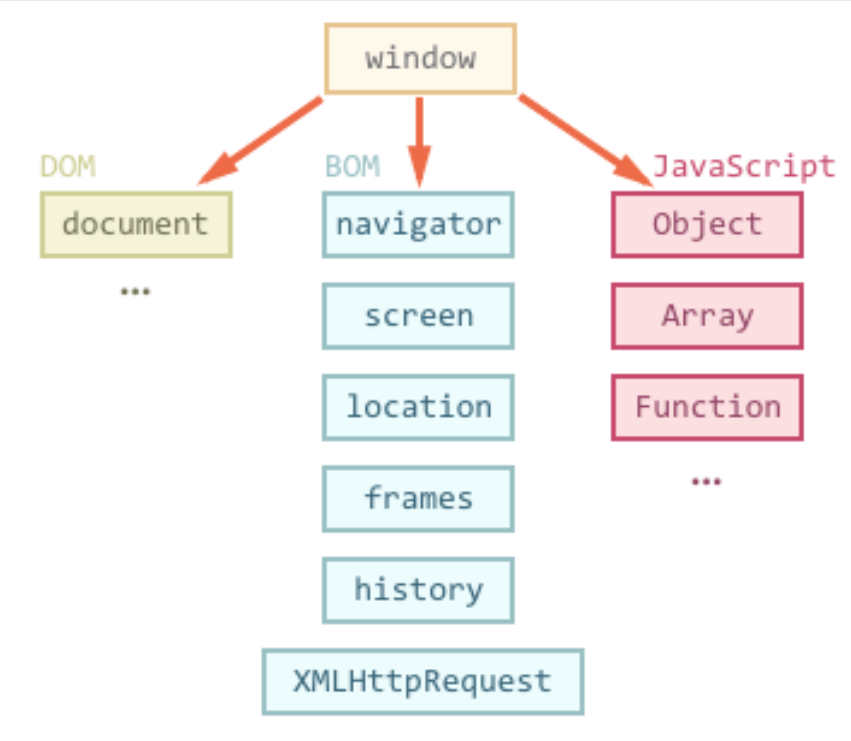
- Reacting to Events
  - A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.
  - To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:
  - onclick=JavaScript
  - Examples of HTML events:
    - When a user clicks the mouse
    - When a web page has loaded
    - When an image has been loaded
    - When the mouse moves over an element
    - When an input field is changed
    - When an HTML form is submitted
    - When a user strokes a key

```
<h1 onclick="this.innerHTML =
'Ooops!'">Click on this text!</h1>
```

```
<h1 onclick="changeText(this)">Click on this
text!</h1>
<script>
function changeText(id) {
  id.innerHTML = "Ooops!";
}
</script>
```

Browser Object Model (BOM)

- ✓ The Browser Object Model (BOM) is used to interact with the browser.
- ✓ The default object of browser is window
- ✓ The figure below shows the relation between DOM and BOM.



Windows Object Model

- The Windows Object Model facilitates the browser windows object to display messages similar to the alert messages of JavaScript.

Windows Object Method	Method Description
alert()	The alert box message is displayed with the OK button in the Windows method.
confirm()	The confirm dialog box message is displayed in the Windows method with the OK and Cancel buttons.
close()	Closes the current window using the Windows method.
open()	The Windows method opens a new window.
prompt()	It asks the user to type in some text. A dialogue box for user input is displayed by the Windows method.
setTimeout()	After a certain time, a Windows method performs actions such as calling functions, evaluating expressions, etc.
setInterval()	It repeatedly calls a callback function with a fixed delay between each call.

Screen Object Model

- The Screen Object Model allows us to retrieve browser data like screen width, height, color depth, pixel depth, etc.

Screen Object Property	Description
width	This method returns the width of the screen.
height	This method returns the height of the screen.
availWidth	This method returns the available width of the screen.
availHeight	This method returns the available height of the screen.
colorDepth	This method returns the color depth.
pixelDepth	This method returns the pixel depth.

History Object Model

- ✓ The History Object Model returns an array of URLs that the user has visited in their browser history.

History Object Method	Description
forward()	This method loads the next page.
go()	This method loads a page with the specified page number.
back()	This method loads the previous page.

Navigator Object Model

- ✓ The Navigator Object Model allows us to retrieve browser information such as the application name, cookie information, version, user language, user agent information, etc.

Navigator Object Property	Description
appName	This method returns the name of the application.
appVersion	This method returns the version of the application.
language	This method returns languages that are supported by Firefox and Netscape browsers.
appCodeName	This method returns the name of the code.
cookieEnabled	This method returns whether the cookies are enabled or not.
userAgent	This method returns the details of the user agent.
userLanguage	This method returns the user language that is supported by the Internet Explorer browser.
systemLanguage	This method returns the system language that is supported by the Internet Explorer browser.
plugins	This method returns the plugins that are compatible with the Netscape and Firefox browsers.
platform	This method returns the windows platform
online	It informs whether the browser is online or not.

Location Object Model

- ✓ In JavaScript, the Location object window allows us to save information about the current page's location(URL).
- ✓ It redirects the browser to a different new page.
- ✓ Although there is no universal standard for the location object, it is supported by all major browsers.

Methods	Description
assign()	This method loads a new web page document.
reload()	Reload the current document using the location.href property.
replace()	The current document is replaced with the new specified document. It is not possible to return to a previous document using the back button on the browser.

BOM Cookies (JavaScript Cookies)

- What are Cookies?
  - Cookies are data, stored in small text files, on your computer.
  - When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user.
  - Cookies were designed to solve this problem "how to remember information about the user":
  - When a user visits a web page, his/her name can be stored in a cookie.
  - Next time the user visits the page, the cookie "remembers" his/her name.
  - Cookies are saved in name-value pairs like below:
    - username = Dagim Sewumehon

...cont'd

- How Cookies Work?
  - When a user sends a request to the server, each request is treated as if it were sent by a different user. To identify returning users, cookies are used.
  - Cookies are small pieces of data sent by the server and stored on the user's browser at the client side. When the server responds to a request, it includes a cookie in the response.
  - The browser automatically attaches the cookie to subsequent requests made by the user. This allows the server to recognize and identify the user based on the cookie.

Create a Cookie with JavaScript

- ✓ JavaScript can create, read, and delete cookies with the document.cookie property.
- ✓ With JavaScript, a cookie can be created like this:
  - ❑ document.cookie = "username=John Doe";
- ✓ add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:
  - ❑ document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC";
- ✓ With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.
  - ❑ document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC; path=/";

End of Ch.6

Questions, Ambiguities, Doubts, ... ???